

面向 AI Agent 轨迹的可配置语义剖析

AgentSight 研究原型

2026 年 7 月 4 日

摘要

AI agent 的执行历史已经不再只是一次 prompt 或一棵 trace tree。一个真实轨迹可能同时包含用户意图、LLM 响应、tool call、API 调用、GUI 动作、浏览器动作、进程、文件、网络、计划步骤、subagent 以及 benchmark 给出的成功、失败、安全、重复和冗余标签。现有 agent observability 工具通常把这些对象固定成 prompt、session 或 span 层级；传统 profiler 则固定成函数或进程层级。这两种做法都会把 profiling 问题过早绑定到某一种边界上，使同一段轨迹很难按任务、阶段、工具、动作、质量标签或人工分组递归折叠。

本文提出一种更小的语义剖析模型：系统只保留两个核心抽象，**operation** 和 **operation stack**。**operation** 是带字段和权重的观测，prompt、session、tool call、process、syscall、GUI action 和 plan step 都只是 operation 的不同形态或字段。**operation stack** 是用户从字段中选择的递归投影，mapping 和 tagging 在 stack 构造前派生字段，而不是创建第三种 profiler 对象。关键点是聚合不是第三个对象，也不是固定 trace tree；聚合由 query time 选择的 stack 形态决定。我们在 Rust agentpprof 中实现该模型，支持 `--view`、`--where`、`--stack`、`--op-map`、`--op-map-file`、`--trace-file`、`--standard-trace-file`、`--deterministic-output` 和可复现实验用的 `--profile-spec`，并把 15 个公开标注 agent 轨迹数据源投影为同一类 operation JSONL。

我们在 15 个轻量采样的公开标注轨迹来源上评估该模型，同步步完整测试集、图片归档或 gated corpus。核心结果显示，Web、mobile、desktop、software、API、tool-dialogue、failure、安全、human group 和 step-quality 轨迹都可以进入同一 operation layer，核心 14 个数据集包含 42,590 个 operations，补充 ScaleCUA 后为 47,590 个 operations。同一批 13,265 个 operations 可以从 9 个 dataset stacks 递归展开到 57 个 phase stacks、226 个 tool/semantic stacks 和 455 个 action stacks，而把固定 session 加入默认层级会膨胀到 3,757 个 stacks。本地 agent session 和 Chrome/Perfetto-style trace 只作为 fixture-level exchange smoke 使用，完整 benchmark conversion、image-heavy corpora 和真实 OpenTelemetry/Perfetto producer 兼容性仍是后续工作。

第二组结果验证 stack 不是 prompt/session/span 的别名。OSWorld-Human 的单动作轨迹可以按人工 grouped-action 边界折叠，保守边界 F1 为 0.627 且 precision 为 1.0。AgentNet 的 1,000 个 Ubuntu 人工桌面任务产生 16,741 个 PyAutoGUI operations，并把 step correctness 和 redundancy 作为普通字段保留下来。Mapping/backend 的可支持范围是 deterministic 或 supervised field derivation：4 个标签家族可训练，但 AgentRewardBench looping 被更简单的 `repeat_signal_change` baseline 完全解释，所以当前证据不支持通用无监督 intent detector。

第三组结果把 value claim 提升为真实标注 trace 上的 profiler localization 和 ranking claim，但仍不声称用户效用。在 AgentRewardBench、SATraj-OS、AgentNet 和 OSWorld-Human 的 4 个数据集、6 个 oracle-backed tasks、34,539 个 operations 和 3,699 个 positives 上，operation-stack packet 比 flat summary 更 selective 的任务为 6/6，含 positive group 的任务为 6/6，含 high-lift evidence 的任务为 5/6。相对 fixed-session，operation-stack selected recall 更高的任务为 5/6，但 selected work 更低的任务只有 2/6，fixed-session 在 4/6 tasks 上 selected work 更低。把 162 个非 oracle view/ranker/budget points 放入 Pareto frontier 后，operation-stack 在 6/6 tasks 上 nondominated，在 4/6 tasks 上取得 best lift，并在 4/6 tasks 上取得 30% work 内 best recall；flat 和 fixed-session 也在 6/6 tasks 上保留 frontier counterpoints。R320 进一步把 profiler 输出当作 ranking/localization result，用隐藏标签计算 precision@k、recall@budget、F1、AUPRC-style AP、nDCG 和 work-to-first-positive。Top-5 query-aware operation-stack profiling 的 median work 为 0.0937，而 flat summary 为 1.0；相对 fixed-session query-aware drilldown，operation stack 在 5/6 tasks 上提高 top-5 recall，并把 median groups 从 285.0 降到 157.5。Query-aware visible ranking 相比 width-only operation-stack ranking 在 6/6 tasks 上提高 AP，但 top-5 F1 和 work 仍暴露 prevalence 与 calibration 反例。R326 进一步说明 rank policy 的收益不只是某一组手调权重：global equal visible features 在 semantic/coarse stack 上分别赢 4/6 和 5/6 tasks，equal-weight task policy 在 8/12 variants 上接近 weighted policy；但 repaired policy 使用 R325 离线评分结果，因此只支持 actionability，不支持已部署的 label-free universal ranker。R327 复用 R300/R324/R326 已提交的 76 个 profile specs 和已提交 operation JSONL，执行 152 次 Rust profiler 调用：去除 JSON generated_at 后，semantic profile hash、samples 和 unique stacks 均为 76/76 稳定；median runtime 为 1.581s，p95 为 2.719s，最大为 4.273s。R328 在同一 workload 上启用 `--deterministic-output`，把 JSON generated_at 和 pprof profile time 固定为可复现值，使 semantic 与 raw-byte determinism 均达到 76/76；该 run 的 median runtime 为 1.578s，p95 为 2.731s。Task-level narrative 进一步显示，SATraj safety 是最强 selective win，AgentNet quality 是 high-lift/low-recall，AgentRewardBench looping 是 prevalence aggregation，side-effect 和 OSWorld-Human boundary 对 ranker 与 depth 敏感。因此本文支持的 claim 是 operation-stack profiling 能在真实标注 trace 上 faithful 地定位、排序和归因任务相关问题，并提供可审计的 accuracy/work/fragmentation tradeoff，而不是人类用户效用、自动异常检测或相对 fixed-session 的无条件支配。

1 问题

AI agent 轨迹的调试问题正在从单次调用回放变成跨任务剖析问题。开发者不只想知道某个 span 什么时候开始和结束，也想知道哪类任务导致最多重复点击，哪些 GUI 轨迹反复输入，哪个工具调用族对应失败，哪些安全攻击类型集中在某些操作阶段，或者一个 prompt 序列能否被折叠成 task、phase、action 和人工 subtask 等多个深度。如果 profiler 把 session、prompt 或 tool call 写死成核心层级，这些问题会被迫使用同一个边界。如果 profiler 只保留进程或函数栈，上层语义和 benchmark oracle 又会丢失。

本文的出发点是把 agent trace 中所有可剖析对象都降到同一层。一个 prompt 可以是 operation，一个 tool call 可以是 operation，一串 prompt 也可以通过 mapping 被派生为同一个 intent 或 task operation field。同样，GUI action、API call、browser action、process event、syscall、安全标签、step correctness 和人工 grouped-action id 都不是新的 profiler 抽象，而是 operation fields。Profiler 的核心任务不是决定唯一正确边界，而是让用户和实验可以在同一批 operations 上选择 stack 字段、派生 mappings，并得到不同深度的递归折叠。

这个问题有两个系统挑战。第一，抽象必须足够小，否则 prompt、session、tool call、plan、subagent 和 process 会各自形成独立 pipeline。第二，抽象又必须足够可配置，否则 flamegraph 只会成为固定视图，不能回答边界、失败、安全和质量诊断问题。本文的目标不是证明某个单一 flamegraph 最好，而是证明 operation 和 operation stack 足以覆盖多类真实标注 agent 轨迹，并且 stack shape 与 aggregation 的选择会系统性改变质量、边界、定位、work 和碎片化分析结果。

2 设计

2.1 两个核心抽象

operation 是唯一的样本级对象。每个 operation 是一个带权重的字段集合，例如 `op=prompt`、`op=llm`、`op=tool`、`tool=browser`、`action=click`、`phase=navigate`、`status=failure` 或 `step_correct=incorrect`。这些字段可以来自本地 Codex/Claude session，也可以来自外部 benchmark 的标注轨迹。实现上，外部轨迹使用每行一个 JSON object 的 operation JSONL，并把原始任务文本、截图和长对话默认排除在提交 artifact 之外。

operation stack 是从 operation fields 中选择出的有序 frame 列表。例如同一批 operations 可以用 `project`、`dataset` 查看数据来源，也可以用 `project`、`dataset`、`task`、`phase`、`op`、`tool`、`action`、`status` 查看动作深度，或者加入 `human_group`、`step_correct`、`safety` 和 `looping` 做诊断视图。Stack 不是 trace tree，也不是 session hierarchy。它是一个查询结果，用户可以按问题改变深度和字段。

2.2 Mapping 和 tagging

Mapping 和 tagging 是派生 operation fields 的一等机制。它们在 stack 构造前运行，读取 operation 的 `key=value` 文本，并写入新的字段。例如 desktop action 中的 `type` 和 `key` 应先被映射到 `phase=input`，再考虑通用 web action verb；API/tool traces 应先进入 tool/API phase family，再处理 `search` 或 `create` 这类词。这一顺序来自跨 web、desktop、API/tool 和 step-quality 轨迹时暴露出的关键设

计约束。如果通用 action rule 抢在 operation-family rule 前面运行，desktop、web 和 API 轨迹会被错误合并。

Mapping 不引入第三个抽象。它只是在 operation 上写字段，使同一个 stack specification 可以跨数据集复用。当前 Rust CLI 支持 `inline --op-map FIELD:LABEL=REGEX` 和文件形式 `--op-map-file`。--where FIELD=REGEX 和 FIELD!=REGEX 在 mapping 之后、stack 构造之前选择 operation 子集，多条 predicate 取 AND；它实现形式化模型里的 ϕ ，不是新的 profiler 对象。script/operation_map_infer.py 可以从已标注 operations 中生成同样格式的规则文件，并用这些规则测试 held-out 和 leave-dataset-out 泛化。--profile-spec 把 operation files、mapping files、predicate、view、stack 和输出路径打包成 JSON 配置，便于 reviewer 复现实验；命令行参数仍可覆盖 spec 默认值。--deterministic-output 或 spec 中的 deterministic_output 可以把 JSON generated_at 和 pprof profile time 固定为可复现值，用于 byte-stable artifact checks。它不改变模型本身，只是把已有 operation 和 operation-stack 查询显式记录下来。

2.3 Views 和非火焰图分析

--view 决定采样和权重，例如 operation count、token、file、network 或 time。--stack 决定递归折叠形状。Folded stack 和 SVG flamegraph 是其中一种输出，但不是唯一输出。我们同时生成 JSON profile、stack tree、top-field table、parent-child transition、depth sweep、boundary F1、V-measure、coverage report、grouped-action report 和 HTML quality report。这些视图共享同一 operation 输入和 stack 语法，因此 failure、safety、looping、redundancy 和 human group 可以被当作普通字段评分，而不是另建 failure profiler 或 safety profiler。

3 实现

agentpprof 是当前被测实现。它是 Rust CLI，读取本地 Codex/Claude 历史或外部 operation JSONL，并输出 pprof-compatible profile、folded stacks、SVG 和 JSON。核心命令形态如下：

```
agentpprof --operation-file ops.jsonl --view operations
--stack project,dataset,task,phase,op,tool,
action,status --op-map-file operation-map.txt
--where task=verify
-o out.folded
```

外部数据集转换器位于 script/agent_trace_datasets.py。转换器只下载或流式读取所需公开文件的采样前缀。对于包含截图、长 prompt 或完整对话的数据集，默认保存 redacted rows 和 operation JSONL，不提交原始数据。本地 agent session 通过 agent-session 解析为 agentsight.agent-session.trace.v1 trace；agentpprof 可以用 --export-trace 写出 filesystem-normalized trace，用 --trace-file 读回，也可以通过 script/agent_trace_convert.py to-operations 转成 operation JSONL。script/agent_trace_exchange_eval.py 将这三步、filesystem/tool-command portability check 和 folded-output equality check 固化为一个可复现实验；该检查不声称 prompt/LLM preview 已完整脱敏。agentpprof 还可以用 --export-standard-trace 写出 Chrome/Perfetto-style traceEvents，再用

--standard-trace-file 导入为普通 operations 后折叠。
 script/agent_trace_convert.py export-standard
 --format chrome 和 import-standard
 --format chrome 保留为显式中间文件脚本；
 script/agent_trace_chrome_exchange_eval.py 验证该路径不改变 folded operation-stack 输出。质量评估脚本位于 script/operation_stack_quality.py，它复用 agentpprof 的 operation mapping contract，并计算字段覆盖率、oracle V-measure 和序列相邻边界 F1。Stack 结构分析脚本位于 script/operation_stack_analysis.py，用于生成 flamegraph 之外的 tree、top-kind 和 transition 报告。

这个实现刻意保留简单接口。它不像 jq 那样完整表达 JSON 查询语言，但它已经把 profiler 的关键自由度暴露为 flags 和可提交配置：数据源由 --trace-file 或 --operation-file 选择，权重由 --view 选择，查询子集由 --where 选择，字段派生由 --op-map 选择，递归边界由 --stack 选择，JSON group 排序由 --rank-rule、--rank-op-rule 和 --rank-mode 选择，复现实验由 --profile-spec 固化。R321 用同一份 R300 真实标注 operation JSONL 验证该实现路径：profile spec 中的 mapping-derived where_rules 分别选出 729/729、714/714 和 4,285/4,285 个预期 operations 后再折叠。R322 进一步让 Rust JSON profile 输出 visible-rule ranked operation-stack groups；在同一 6 个标注任务上，rank_rules 相比 width ranking 的 AP 提升覆盖 4/6 tasks，top-5 lift 提升覆盖 3/6 tasks，同时 SATraj 和 side-effect 反例显示二值 stack-text boost 仍弱于 R320 的 group-feature query-aware ranker。R323 把该反例转化为 rank-mode 机制隔离：rule-score 相比 width-boost 的 AP 提升覆盖 4/6 tasks，top-5 lift 提升覆盖 4/6 tasks，并把 SATraj first-positive work 从 0.6376 降到 0.0842；side-effect 和 OSWorld-Human 仍保留为 depth/ranker counterexamples。R324 把 per-operation visible feature density 接到 Rust profiler：脚本先从 R300 source 派生删除 oracle fields 的 visible-operation JSONL，再让 rank_op_rules 匹配 mapping/filtering 后的单个 operation field=value token，并在 folded group 内聚合 matched operation weight；semantic stack 上 AP 相比 width 提升 5/6 tasks、top-5 lift 提升 4/6 tasks、first-positive work 改善 5/6 tasks，coarse stack 也在 AP 上提升 4/6 tasks，同时保留 OSWorld-Human 反例。R325 在同一 Rust profile-spec 路径上做 leave-one-feature ablation：它发现 7 个 critical feature instances、3 个 misleading feature instances，并显示 coarse stack 只在 2/6 tasks 上 AP 更好但在 6/6 tasks 上减少 group 数。R326 进一步测试 rank-policy robustness：global equal visible feature bank 在 semantic/coarse stack 上 AP 相比 width 分别赢 4/6 和 5/6 tasks，task-equal 在 8/12 task/depth variants 上与 weighted task policy 的 AP 差距不超过 0.02，R325-guided repair 在 2/3 misleading-feature cases 上改善 AP、在 2/3 cases 上改善 first-positive work，其中 1/3 同时改善两者；repair 是 post-hoc actionability check，不是 label-free deployment policy。R327 再把这些可提交 profile specs 作为 reproducibility/cost workload：R300 的 4 个 view specs、R324 的 12 个 rank-feature specs 和 R326 的 60 个 robustness specs 均由 Rust agentpprof --profile-spec 重复执行两次；semantic profile hash 全部稳定，raw-byte hash 只有 4/76 稳定，因为 JSON profile 包含生成时间戳。R328 在同一 76 个 specs 上启用 --deterministic-output，使

raw-byte hash 也达到 76/76 稳定；这关闭的是 artifact byte-stability 缺口，不是新的 accuracy benchmark。因此 prompt、session、toolcall、process、syscall、plan 和 sub-agent 都可以在同一套 mechanism 中出现，而不是成为互不兼容的 object model。

4 实验方法

评估遵循八个原则。第一，只使用公开的、别人已经标注好的 agent 轨迹或交互轨迹。我们使用 Hugging Face Dataset Viewer、HF repo JSONL、公开 GitHub JSON 和公开 benchmark artifacts 进行轻量采样，不同步完整测试集，也不提交原始外部样本。第二，每个 claim 都必须对应可执行 oracle。对于 phase/action 结构使用 V-measure 和相邻边界 F1；对于 grouped-action 使用人工 group id；对于 looping、safety、step correctness 和 redundancy 使用数据集原生标签；对于递归深度使用同一批 operations 下 unique stack 和 compression 变化。第三，负结果保留在论文中。例如 AgentNet 的 task status 很弱地预测 per-step correctness，repeat signal 也很弱地预测 step redundancy。这说明 profiler 能承载这些诊断字段，但简单 mapping 不能替代专门质量模型。第四，论文 claim 必须从 artifact 机械回溯。Claim synthesis、reviewer evidence packet、paper value/novelty synthesis、paper evidence matrix 和 submission audit 都只读取 tracked/clean result files，并输出 claim verdict、paper-ready wording、must-not-claim 和 source paths。这些文件不是新实验，也不是 profiler object。它们的作用是防止摘要、结果、局限和结论在数字、scope 或 unsupported claim 上漂移。第五，boundary backend 必须仍然服从两个抽象。监督式 adjacent-boundary backend 只写回 learned_group_pattern、learned_segment_pattern、learned_segment_position 和 learned_boundary_prev 等 operation fields。递归折叠仍由 agentpprof 使用普通 --profile-spec 和 --stack 完成。每个候选标签家族还必须先通过 suitability 和 simple-baseline gate，防止把 safety、history context 或 repetition proxy 错当作语义边界。第六，真实问题价值必须先通过可复现 proxy 任务收敛。我们从 AgentRewardBench、SATraj-OS、AgentNet 和 OSWorld-Human 的已有 operation JSONL 构造 6 个 oracle-backed analysis tasks，比较 flat、fixed-session、semantic operation-stack 和 label-drilldown 四种 stack specs。指标包括 positive lift、覆盖 50% positives 的 inspection fraction、group count、top-group session support、selected recall 和 selected work。这仍不是人类用户实验，但它把“profiler 是否解决真实问题”从叙述变成可审计的 proxy result。第七，默认浏览、ranking 和 case evidence 都必须避免答案泄漏。Visible packets 不包含 oracle-positive 字段，hidden answer key 单独保存 positives。非 oracle rankers 只读 status、repeat_signal、phase、action 和 environment 等可见字段，不读 looping、safety、step_correct 或 target_positive。因此 first-positive、first-enriched 和 high-lift evidence 只是对同一 operation/operation-stack packet 的隐藏标签回放评分，而不是 automatic detection 或 human utility。第八，R320 把 profiler 输出直接当作 ranking/localization result。它在同一 6 个真实标注任务上比较 flat、fixed-session、dataset-native hierarchy、raw action stack、operation-stack、label-drilldown oracle view，以及 width、visible-risk、query-aware 和 oracle-upper-bound rankers。非 oracle policies 只使用可见 operation

fields, 隐藏标签只用于 scoring。指标包括 precision@k、recall@operation-budget、F1@k、AUPRC-style average precision、nDCG、work-to-first-positive、groups-to-50% recall、group count 和 task-level optimization insight。这个设置把 profiling paper 的主 claim 从“analyst 是否更快”改为“profiler 自身是否能在真实 workload 上 faithful 地定位、排序并指导优化”。

4.1 Hierarchy ablation baselines

本文比较的是 hierarchy choices, 而不是把每个已有 observability 产品完整复刻一遍。Flat profile 把每个任务压成一个 summary packet, 用来检验 operation stack 是否比无层级摘要更 selective。Fixed-session drilldown 把 session 或 demo id 放进 stack, 用来模拟 session/prompt-first 调试界面和传统 trace-tree drilldown 的碎片化风险。Prompt/session tree、tool-call hierarchy 和 OTel/span-like trace tree 在本实验中都被建模为不同 stack specs: 它们仍然读取同一批 operations, 只是把 session、prompt、tool、process 或 trace id 放在更靠前的 frames。Label-drilldown 是 oracle-aware upper-bound style view, 它把 hidden label 直接放入 stack, 用来界定“如果答案字段已经可见”时的上限, 而不作为真实用户界面 baseline。这些 ablations 的目的, 是隔离边界选择对聚合、selectivity、recall 和 inspected work 的影响。因此本文不会声称 agentpprof 支配所有生产 observability UI, 只声称 operation stack 把这些边界选择统一成可配置查询。

5 结果

5.1 RQ1: 两个抽象是否覆盖多类轨迹?

核心 14 数据集产生 42,590 个 operations 和 1,497 个 unique stacks, compression ratio 为 28.45。加入 ScaleCUA 补充样本后, 总量为 47,590 个 operations 和 1,611 个 unique stacks, compression ratio 为 29.541。这些 operations 覆盖 web、mobile、desktop、software、API、tool-agent-user dialogue、expert failure labels、safety labels、human grouped-action labels 和 desktop step-quality labels。所有这些标签都以 operation fields 形式进入同一 pipeline。没有任何数据集需要新增 prompt、session、GUI、safety 或 quality-specific profiler object。

这支持 C1 的机制性版本: agentpprof 可以把异构 agent 轨迹统一为 operations, 并用用户指定 stack 折叠。它不支持所有 agent 数据都容易转换。适合该方法的轨迹通常具备四个特征: 有有序 step 或 turn, 有稳定 session/task id, 有动作或阶段标签, 有 outcome、quality、group 或 safety 等较高层 oracle。只有静态截图 grounding、缺少顺序、缺少 action label、强依赖大图片归档或 gated access 的数据源更适合作为后续扩展, 而不是当前主证据。

5.2 RQ2: 递归 stack 是否比固定边界有价值?

R286 在同一批 13,265 个 operations 上扫过 8 个 stack depths。只保留 dataset 时有 9 个 stacks; 加入 task 后为 11 个; phase depth 为 57 个; tool/semantic depth 为 226 个; action depth 为 455 个; 加入固定 session 后膨胀到 3,757 个。相对于 dataset depth, 固定 session 的 unique stacks 增加 417.4 倍。这个结果直接反驳把 session 或 prompt 作为默认 profiling 边界的设计。Session 是有用 drilldown field, 但如果默认写在 stack 里, 它会把可聚合的行为切碎。

早期 3,797-operation set 上也出现同样信号。Flat/mapped stack 为 103 个 unique stacks, 而固定 demo/session stack 为 1,083 个 unique stacks。这说明固定边界会带来约 10.5 倍碎片化, 而 mapped operation stack 可以保留聚合同时增加语义深度。因此论文中的 novelty 不是“画 flamegraph”, 而是把 agent profiling 的边界选择变成 operation-stack query。

Profile spec 把这个 query 自由度变成可复现实验配置。同一个 AgentNet spec 引用相同 operation JSONL 和 op-map 文件, 默认产生 608 个 diagnostic stacks; 命令行只覆盖 --stack 和输出路径后, 样本数保持 16,741 不变, unique stacks 降到 83。这说明 stack 深度是可审计、可覆盖的查询参数, 而不是写在 prompt/session 或某个可视化中的层级。

这种可配置性不只改变图的形状, 也改变 reviewer 能验证的问题。同一套 outputs 同时给出 depth sweep、stack tree、transition/top-field、quality report、grouped-boundary report 和 history-depth report。这些报告回答同一个机制问题: 一个 profiler 是否可以把边界选择延后到查询时, 而不是在采集时固定为 prompt、session 或 span tree。当前证据支持三条 scoped 结论。第一, 异构轨迹和本地 session 可以先进入同一 operation layer, 再用用户选择的 stack 折叠。第二, recursive stacks 能表达 dataset、task、phase、tool、action、human-group、safety 和 quality-label 等多种深度, 但不能声称恢复所有真实意图边界。第三, field derivation 是可扩展接口, 因为 mapping 和 supervised backend 只写入 operation fields, profiler 仍按 operation stack 折叠。这些机制结果给出 paper novelty, 但 user-utility、通用 boundary detector 和完整 trace-platform compatibility 仍是后续 gate。

真实问题 proxy 把“定位 looping、side-effect、unsafe、incorrect、redundant 和 group-start positives”变成 6 个自动化分析任务, 并比较 flat、fixed-session、semantic operation-stack 和 label-drilldown。Visible packets 不含 oracle-positive 字段, hidden answer key 单独保存 positives; query-aware rankers 只读可见字段。这些任务显示 operation-stack 是 flat summary 与 fixed-session drilldown 之间的中间视图: 它远比 flat selective, 相比 fixed-session 通常有更高 selected recall 和跨 session 聚合, 但并不总是消耗更少 inspected operations。Paper evidence matrix 和 reviewer-stress audit 因此把 C4 固定为 automated inspectability tradeoff: operation-stack 在 6/6 tasks 上比 flat 更 selective, 6/6 含 positive group, 5/6 含 high-lift evidence, 5/6 selected recall 高于 fixed-session; 反例是 selected work 只在 2/6 tasks 低于 fixed-session。因此 C4 不能写成 user-utility、automatic detection 或 baseline-dominance claim。

R313 将同一批 visible-field rankers 和 case-packet baselines 放进 Pareto frontier, 而不是只做成对比较。Frontier 同时优化 recall、lift、inspected work 和 inspected groups。结果是 operation-stack 在 6/6 tasks 上都 non-dominated, 并在 4/6 tasks 上给出最高 non-oracle lift、在 4/6 tasks 上给出 30% inspected-work budget 内最高 recall。但 flat 和 fixed-session 也分别在 6/6 tasks 上保留 frontier points。这使 C4 更像一个 systems claim: profiler 的价值是把 view、ranker 和 stack depth 暴露成可配置分析 surface, 而不是替用户固定一个永远最优的层级。

表 1: 公开标注轨迹数据源和当前用途。ScaleCUA 是补充点, 主结论主要依赖前 14 个 oracle-rich 数据源。

数据源	原生标注	转换方式	主要用途
WebLINX, Mind2Web, AgentTrek, WebShop API-Bank, ToolBench, tau-bench	web action, demo/task, reward 或 action history	HF Dataset Viewer 或 HF repo shard	web navigation 和 shopping trajectories 的跨源 smoke。
SWE-agent	API/tool call, solution path, 多轮 user/assistant/tool messages, outcome	HF rows 或 repo JSONL	tool/API/dialogue phases 和 user-agent-tool operation 形态。
AndroidControl, GUI-Odyssey	issue trajectory, command action, success target	HF Dataset Viewer	software-agent command 轨迹。
AgentRewardBench	mobile/cross-app step action 与 instruction	public/HF samples	移动 GUI action depth 和跨 app scale。
SATraj-OS	expert success, side-effect, looping, optimality	annotation rows 加 cleaned BrowserGym steps	failure, looping, side-effect 的非 flame-graph 诊断。
OSWorld-Human	desktop action, success, safety, reward, attack type	HF safety config	桌面安全和 attack-type operation fields。
AgentNet	human single-action 和 grouped-action 轨迹	GitHub JSON files	人工 grouped-action 边界 oracle。
ScaleCUA	human desktop PyAutoGUI, task outcome, step correctness, redundancy	HF repo JSONL streaming	最大 human desktop step-quality oracle。
	GUI navigation action, previous-operation context, gold action	HF annotation JSONL streaming	补充多步 GUI history-depth smoke。

表 2: R311 task-level reviewer-stress audit. OS 表示 operation-stack top-5 packet, FS 表示 fixed-session top-5 packet; work 和 recall 都是 selected packet 覆盖比例。

Task	数据集	Query family	OS work	OS recall	OS lift	FS recall	主要结论和反例
incorrect step	AgentNet	step-quality	0.0014	0.0034	2.406	0.0126	很小 work 下有 high lift, 但 selected recall 极低且低于 FS。
redundant step	AgentNet	step-quality	0.0089	0.0177	1.984	0.0123	OS recall/lift 更高, 但 FS work 略低且更早命中 positive。
looping	AgentRewardBench	failure	0.4938	0.6508	1.318	0.2381	Positives 过于普遍, OS recall 高但没有 $\geq 1.5\times$ high-lift group。
side effect	AgentRewardBench	failure	0.1454	0.1139	0.783	0.0000	OS 找到 positives 且 work 低于 FS, 但 top-5 lift 低于 prevalence。
group start	OSWorld-Human	boundary	0.4074	0.3874	0.951	0.0327	OS recall 高但 work 大, ranking depth 决定是否有 high-quality packet。
unsafe operation	SATraj-OS	safety	0.0420	0.2621	6.238	0.0579	最强 selective win, OS recall/lift 明显高于 FS, 但 FS first-positive work 更低。

5.3 RQ3: label-derived mappings 在什么条件下迁移?

本节支持的是 configurable deterministic/supervised field derivation 在合适标签家族上的迁移, 而不是通用无监督边界发现。

R281 从 13,265 个 labeled operations 中生成 10 条 mapping rules, 复现手写 mapping 的 folded 输出。R282 使用 dataset-stratified held-out split, 在 9,275 个 train operations 上生成规则, 并在 3,990 个 held-out operations 上评估。Mapped stack 产生 209 个 held-out stacks, compression ratio 为 19.091; 无 mapping baseline 为 284 个 stacks, compression ratio 为 14.049。Task/dataset V-measure 从 0.837 提升到 0.853。Phase/action boundary F1 为 0.777, precision 为 1.0, 说明当前 mapping 是保守的。它不会制造 false positive phase changes, 但会合并一些 fine-grained action changes。

R283-R285 进一步做 leave-dataset-out。在修正 API/tool phase precedence 后, R285 对 9 个 held-out datasets 中的 6 个减少 unique stacks, 0 个出现 negative stack reduction, weighted stack reduction 为每 1,000 operations 1162.005。这说明 mapping 不是单个数据集的 ad hoc pretty printer。它仍然是 deterministic taxonomy-seeded mapping, 不是无监督边界发现。R297 开始测试更强的 boundary backend。它在 OSWorld-Human exact-aligned sessions 上按 session 做 deterministic split: 191 个 train sessions 产生 2,655 个 train adjacent pairs, 96 个 test sessions 产生 1,036 个 test adjacent pairs。模型使用非 oracle operation fields 训练 Bernoulli adjacent-boundary predictor, 明确排除 human_group、group_index、

group_position、group_size、group_pattern 和 group_alignment。在 held-out pairs 上, learned backend 的 precision 为 0.7400, recall 为 0.8102, F1 为 0.7735; phase-change baseline 为 0.2919, action-change 为 0.5142, conservative group_pattern reference 为 0.5810。随后脚本把预测边界写成 learned_group_pattern 和 learned_group_position 字段, Rust agentpprof 在 1,132 个 held-out operations 上折叠得到 74 个 stacks。当前可支持的 claim 是 configurable deterministic/supervised field derivation 能在 held-out 设置下改善语义聚合; 完整 model-backed 或 unsupervised boundary detector 仍是后续工作。

R299 将这个 backend 接口复制到现有标注家族, 并加入 suitability/calibration gate。脚本检查 OSWorld-Human human-group、AgentNet step correctness、AgentNet step redundancy、AgentRewardBench looping、SATraj safety、ScaleCUA history state 和 tau-bench tool-dialogue role 共 7 个候选。其中 SATraj safety 在当前样本中没有 session-local adjacent positives, ScaleCUA history state 是 context marker 而不是语义边界, tau-bench 在 R287 没有 tracked operation JSONL, 因此不训练。四个可训练候选的 held-out learned F1 分别是 OSWorld-Human 0.6916、AgentNet step-correct 0.3197、AgentNet step-redundant 0.3361 和 AgentRewardBench looping 0.7833。AgentNet 两个质量状态边界都优于 always-boundary baseline (0.2155 和 0.2645), 但 precision 低且 calibration error 仍明显; AgentRewardBench looping 虽然 learned F1 达到 0.7833, 却被 repeat_signal_change baseline 的 1.0 F1 完全解释。这把 C3 的范围收得更清楚: backend 可以统一地派生 stackable operation fields, 但每个标签家族都必须证明它真的是边界

oracle, 并且必须胜过简单 sequence-derived fields。

5.4 RQ4: 能否恢复真实人工边界?

OSWorld-Human 是当前最强的 grouped-boundary oracle。R290 转换全部 369 个 human desktop tasks, 得到 6,010 个 single-action operations。转换器把 grouped-action 序列展平后与 single-action 序列对齐, 只有 exact-aligned tasks 才写入 human_group、group_pattern 和 group_position 字段。最终 320 个 tasks、4,011 个 operations 和 2,075 个 session-local groups 进入 grouped-boundary scoring; 31 个 content-mismatch tasks 和 18 个 length-mismatch tasks 保留用于 action profiling, 但不被当作 gold group oracle。

在 OSWorld-Human 上, 普通 action-depth stack 有 482 个 unique stacks, grouped-depth stack 降到 106 个。Phase/action boundary F1 为 0.638, precision 为 1.0。更重要的是, 保守的 group_pattern 对人工 human_group boundary 的 F1 为 0.627, precision 为 1.0, recall 为 0.456。这不是完美边界 detector, 但它给出有意义结论: operation stack 可以在同一条单动作序列上选择 action-depth 或 group-depth, 并以人工 grouped-action 标签作为 oracle 验证。Prompt 或 session 没有参与这个抽象。

5.5 RQ5: profiler 能否 faithful 地定位和排序真实标注问题?

AgentRewardBench、SATraj-OS 和 AgentNet 说明 operation stack 不局限于 flamegraph 热点。AgentRewardBench 的 729 个 expert-reviewed web-agent operations 全部携带 success、side-effect、looping 和 optimality 字段。Action-derived repeat_signal 对 expert looping label 的 V-measure 为 0.378, 而单步 step_error 对 looping 的 V-measure 只有 0.011。这说明 sequence-derived operation fields 可以捕捉一部分真实 failure mode, 也说明简单错误标签不能替代序列诊断。

SATraj-OS 的 4,285 个 desktop computer-use operations 全部携带 safety、attack type、status 和 reward-related fields。其中 622 个 operations 标为 unsafe, attack type 覆盖 account、induced-text、popup、OS 和 unknown-file。这些标签与 phase/action 关系较弱, 例如 attack-type/action V-measure 只有 0.093。这个负结果是有价值的: 安全攻击类别不是 action taxonomy 的简单函数, 因此应该作为独立 operation field 被折叠和过滤, 而不是强行塞进 phase。

AgentNet 是当前最大的 human desktop step-quality oracle。R291 采样 1,000 个 Ubuntu tasks, 得到 16,741 个 PyAutoGUI operations。所有 operations 都有 task outcome、alignment score、efficiency score、difficulty、step correctness 和 redundancy 字段。Step correctness 覆盖 13,844 correct、874 incorrect 和 2,023 unknown; step redundancy 覆盖 9,334 necessary、733 redundant 和 6,674 unknown。Task status 对 step correctness 的 V-measure 只有 0.015, repeat signal 对 step redundancy 的 V-measure 只有 0.010。这说明 profiler 的价值不是用一个粗标签预测所有质量问题, 而是把质量 oracle 保留为可折叠、可 drilldown、可评分的 operation fields。

R300 把这些诊断目标组织成 6 个自动化 real-problem proxy tasks。任务覆盖 AgentRewardBench looping 和 side-effect、SATraj unsafe、AgentNet incorrect/redundant steps, 以及 OSWorld-Human group starts, 总计 34,539 个 operations。在同一 operation JSONL 上, Rust agentpprof 的 flat profile 只有 6 个 stacks, fixed-session profile 有 2,012 个

stacks, semantic operation-stack profile 有 944 个 stacks, label-drilldown profile 有 318 个 stacks。按 oracle-sorted clustering quality 评价, semantic operation stack 相比 flat summary 的 median top-positive lift 为 5.726x, 覆盖 50% positives 的 median inspection fraction 从 1.0 降到 0.2879。相比 fixed-session stack, semantic stack 的 group count ratio 为 0.554, top groups 的 session support ratio 为 5.5, 说明它更适合跨轨迹诊断; 但 inspection fraction ratio 为 1.302, 说明 fixed-session drilldown 在部分 instance-local tasks 上仍更快。这支持的是 inspectability 和 aggregation claim, 不是人类效率 claim。

R301 用更严格的 label-hidden analyst-task proxy 复核 R300。它把 visible packet 和 hidden answer key 分开, 默认只按 group width 排序。在 30% inspected-operation budget 下, operation-stack 的 median positive recall 为 0.336, 检查 4.5 groups; fixed-session 为 0.284, 检查 25.5 groups。在 top-10 width-ranked groups 下, operation-stack 的 median recall 为 0.641, 而 fixed-session 为 0.195。这个结果说明 operation stack 能把跨 session 的问题压缩成更少可检查单元, 但它并不保证默认宽度排序总是最省 operation budget。

R302 进一步比较 label-hidden ranking policies。在 top-10 groups 下, query-aware operation-stack ranking 只检查 0.116 operation fraction, positive lift 为 1.587; width ranking 检查 0.671 operation fraction, lift 为 1.079。在 30% operation budget 下, query-aware ranking 把 operation-stack recall 从 0.340 提高到 0.390, 但检查 groups 从 4.5 增加到 39.5。这说明 profiler 可以支持不同分析 policy 的 precision/recall tradeoff, 但这些 policy 仍是 visible-field heuristics, 不是 learned detector。

R304 将 R302 的 ranking policy 变成 reviewer-facing case packet。每个任务取 top-5 query-aware operation-stack groups, visible packet 不包含 oracle fields, hidden answer key 单独保存 positives。在 30 个 case groups 上, median inspected operation fraction 为 0.0937, median positive recall 为 0.188, median positive lift 为 1.6509。SATraj unsafe cases 最集中, work fraction 为 0.042, recall 为 0.262, lift 为 6.238; AgentNet quality cases 的 recall 很低, 但在很小 work fraction 上仍有 1.98 以上 lift。这个结果适合作为 case-study evidence, 而不是 detector evidence。

R305 对同一 case-packet policy 加入 flat 和 fixed-session baseline。Flat view 只有每个任务一个 packet, 因此 median recall 为 1.0, 但必须检查 1.0 operation fraction。Fixed-session view 的 median work fraction 为 0.0163, recall 为 0.0226, lift 为 1.6615。Operation-stack view 的 median work fraction 为 0.0937, recall 为 0.188, lift 为 1.6509。相对 fixed-session, operation-stack 的 median recall ratio 为 3.63, lift ratio 为 1.268, 但 inspected-work ratio 为 1.717。这支持“operation stack 是 flat 与 fixed-session 之间的可配置分析折中”, 而不是“operation stack 在每个任务上都更省 work”。

R320 把前面的 case-packet proxy 升级为 profiler accuracy benchmark。它不再只报告 selected recall/lift, 而是把每个 view/ranker 的 hot groups 当作 ranked localization results, 用 hidden labels 计算 precision@k、recall@budget、F1、AUPRC-style AP、nDCG 和 work-to-first-positive。比较对象包括 flat、fixed-session、dataset-native hierarchy、raw action stack、operation-stack、label-drilldown oracle

view, 以及 width、visible-risk、query-aware 和 oracle-upper-bound rankers。非 oracle policies 只使用可见字段; label-drilldown 和 oracle-upper-bound 只作为上界。表 3 给出主要 policies 的 median 结果。

Operation-stack query-aware profiling 在 top-5 groups 中只检查 median 0.0937 operation fraction, 而 flat summary 必须检查 1.0。相对 fixed-session query-aware drilldown, operation-stack top-5 recall 在 5/6 tasks 上更高, 并把 median group count 从 285.0 降到 157.5。它并不在所有 accuracy 指标上支配其他 hierarchy。Dataset-native hierarchy 的 median top-5 recall 达到 0.8665, 但 work 高达 0.8539; fixed-session 的 work-to-first-positive 只有 0.0044, 但 top-5 recall 只有 0.0233。这正是 profiling paper 需要的 claim: operation-stack 提供更好的 Pareto tradeoff 和更少碎片化, 而不是替代所有 drilldown 视图。

R320 还把结果转成 actionable optimization insight。SATraj unsafe 的 best visible policy 是 operation-stack query-aware, 说明 `environment, phase, action` 与 risky/write-action ranking 对安全定位有效。AgentRewardBench looping 中, `repeat_signal` mapping 明显改善 top-5 F1, 但 positives 过于普遍, 需要 prevalence-aware ranking。Side-effect 和 OSWorld-Human boundary 留下 oracle AP gap, 说明当前 ranker 和 stack depth 需要为 side-effect writes 与 boundary-derived fields 分别调参。AgentNet step-quality 上 raw action stack 和 fixed-session drilldown 仍有优势, 说明 quality problem 需要先定位到可疑 action family, 再进入 session-level examples。R322 将其中一个 ranker surface 放入 Rust profiler 本体: 它证明 visible rank policy 可以作为 JSON operation-stack projection 被复现和评分, 也明确暴露 SATraj safety 对 group-level feature density 的需求。R323 进一步说明 ranking policy 本身是可操作的优化旋钮: 当排序从 width-dominated 切换到 score-first, SATraj 和 AgentNet incorrect-step 的 first-positive work 明显下降, 但 side-effect 与 OSWorld-Human 的 top-5 lift/recall 下降, 提示它们需要更深的 side-effect mapping 或 boundary-derived fields。R324 则把 group-level feature density 作为 Rust profile-spec policy 实现出来: Rust 只读取 scrubbed visible-operation profiler input, hidden labels 不传给 Rust, 只用于 offline scoring; semantic stack 的 operation-feature ranking 在 AP 上赢 5/6 tasks, 在 first-positive work 上赢 5/6 tasks; coarse stack 的 group 数从数百级降到几十级时仍在 4/6 tasks 上提升 AP。R325 进一步把 actionability 具体化: 删除 `repeat_signal` 会显著伤害 looping 定位, 删除 `write-action` 会伤害 side-effect 定位, SATraj 的 `status=success` 是关键排序信号, 但 SATraj 的 loop-like 规则和 OSWorld-Human 的 input-phase 规则会误导排序。R326 再检查这些结论是否过度依赖手调权重: global equal visible feature bank 在 semantic/coarse stack 上分别有 4/6 和 5/6 AP wins, equal-weight task policy 在 8/12 variants 上接近 weighted policy, R325-guided repair 在 2/3 misleading-feature cases 上改善 AP, 在 2/3 cases 上改善 first-positive work, 其中 1/3 同时改善两者。R327 则检查 profile-spec 本身是否可复现、成本是否可报告: 所有 76 个 profile specs 在重复运行中保持 semantic profile content、samples 和 unique stacks 稳定, median 每个 spec 运行时间为 1.581s, p95 为 2.719s; 默认 JSON profile 的 raw-byte determinism 只有 4/76, 因为输出带生成时间戳。R328 再启用 deterministic artifact mode, 固定 JSON `generated_at` 和 pprof profile time; 同一 76

个 specs 的 semantic 与 raw-byte determinism 均为 76/76, median 每个 spec 运行时间为 1.578s, p95 为 2.731s。这些结果支持 artifact/reproducibility claim, 但不等价于 live capture overhead。这说明 profiler 输出不仅告诉我们哪里有问题, 还告诉我们哪类 stack depth、field mapping 和 rank policy 值得调。这些结论是 profiler 输出对优化方向的归因, 而不是人工 analyst 的主观解释。

6 哪些数据集最适合

从当前 15 个方案看, 最适合支撑论文主 claim 的不是“最大”数据集, 而是同时具备顺序、动作、边界或质量 oracle 的数据集。第一类是 OSWorld-Human, 因为它同时给 single-action 和 grouped-action, 是验证递归边界的最直接 oracle。第二类是 AgentNet, 因为它规模大、人工桌面任务多, 并带 per-step correctness/redundancy。第三类是 AgentRewardBench 和 SATraj-OS, 因为它们把 profiler 从 action taxonomy 推到 failure、looping、安全和 side-effect diagnostics。第四类是 GUI-Odyssey/tau-bench/ToolBench 这组跨域补充, 它们覆盖移动 GUI、user-agent-tool dialogue 和 API/tool traces, 证明 operation abstraction 不局限于桌面。

ScaleCUA 是有用补充, 但不是当前主证据。R292 流式采样 5,000 个 Ubuntu navigation rows, 覆盖 131 个 sessions, 最大 step 为 48。该子集的动作 taxonomy 主要是 click 和 terminate, 所以 phase/action 指标达到 1.0 并不表示强泛化。它更适合证明 multi-step GUI history context 可以被投影为 `history_state` 和 `history_depth` fields, 而不是证明复杂边界 detector。

7 相关工作

Profiling, flamegraphs 和 trace analysis. 传统 flamegraph 和 pprof 路线把运行时调用栈折叠成热点视图 [1,16,17]。pprof 不只是固定 call stack: 它支持 sample tags, 也支持用 `tagroot/tagleaf` 把 tag value 作为 pseudo stack frame 可视化 [17]。Perfetto 则代表系统 trace ingestion、SQL trace analysis、trace summary 和 derived events 的成熟方向 [18]。因此本文不能把 novelty 写成“只有我们支持 query-time aggregation”。本文复用 folded-stack 表示, 但 scoped novelty 是 agent-operation record model、recursive multi-field operation-stack projection 和真实标注轨迹上的 hidden-label localization benchmark 这三者的组合。Stack frame 来自 agent operation fields, 而不是语言运行时函数调用或通用 trace SQL schema。因此同一批 operations 可以被折叠为 task、phase、tool、action、quality 或 safety 视图, 也可以改成 fixed-session 或 flat baseline。区别不在图形样式, 而在 agent 语义对象、递归多字段 stack projection 和 R320 的跨数据集 hidden-label scoring。

Agent tracing 和 LLM observability. OpenTelemetry GenAI 和 OpenInference 标准化 GenAI spans、LLM calls、agent reasoning steps、tool invocations、retrieval operations、MCP 和 provider-specific telemetry [2,11]。LangSmith、Langfuse、Phoenix 和 AgentOps 等系统进一步把 LLM call、tool call、latency、token、trace tree、session、observation、evaluation score、goal、plan 和 agent artifact 放进单次 workflow inspection 与生产监控 [12,13,14,15]。这些系统是本文最接近的同问题相关工作。因此本文把 trace/span/session 明确当作 exchange container 或 baseline shape, 而不是加成第三个 profiler 抽象。R320 实际

表 3: R320 profiler accuracy benchmark. Hidden 表示 policy 是否读取 oracle label; WTFP 是 work-to-first-positive. 非 oracle policies 只读 visible operation fields, hidden labels 只用于 scoring.

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	no	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	no	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	no	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	no	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	no	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	no	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	yes	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	yes	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

表 4: R327 profile-spec 可复现性与离线执行成本。Runtime 为 Rust binary 已存在后的每个 spec 秒数; semantic hash 去除 JSON generated_at 字段, raw-byte hash 单独报告。

Workload	Specs	Med. s	P95 s	Med. stacks
R300 views	4	3.448	4.273	631.0
R324 rank	12	1.539	2.692	41.0
R326 robust	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

表 5: R327/R328 在同一 76 个 profile specs 上的 determinism 检查。R328 使用 --deterministic-output。

Run	Mode	Sem.	Raw	Med. s	P95 s
R327	default	76/76	4/76	1.581	2.719
R328	deterministic	76/76	76/76	1.578	2.731

评估的是 fixed-session drilldown 作为 span-tree proxy; 真实 OpenTelemetry/OpenInference/Phoenix span-tree import 仍是后续互操作 baseline, 不是当前已完成结果。本文关注跨轨迹 semantic profiling, 把异构标注轨迹或本地 agent 历史统一为 operations, 并允许用户选择 stack 深度、mappings、tagging 和 ranker。R314 专门检查这些 closest systems 是否进入 novelty map 和 baseline guard。

Computer-use 和 tool-use benchmarks. WebLINX、Mind2Web、GUI-Odyssey、OSWorld/OSWorld-Human、OpenCUA/AgentNet、SATraj-OS、AgentReward-Bench、ToolBench 和 tau-bench 为本文提供真实标注轨迹 [3–10,19–22]。本文不提出新 benchmark。贡献是把这些 benchmark 的动作、质量、安全和人工 group 标签统一为 operation fields, 并用同一个 profiler 机制比较它们适合回答哪些 profiling 问题。这些 benchmark 是 evidence source 和 native-sequence baseline, 不是被本文替代的对象。

Novelty 边界。因此本文的新意不是“第一个 agent observability 系统”、不是“第一个 agent trajectory benchmark”、也不是“第一个 folded-stack 可视化”。当前可 defend 的 scoped novelty 是: 只用 operation 和 operation stack 两个 profiler 抽象, 把 trace/session/span 作为输入容器或 baseline, 在真实公开标注轨迹上把 profile groups 当成 ranking/localization outputs, 并用 hidden labels 计算 accuracy/work、fragmentation 和 actionability。更具体地说, 新意不是 query-time aggregation 本身, 而是把 agent-operation record、recursive multi-field stack shape 和 hidden-label localization benchmark 绑在一起: 同一批 agent operations 因 stack shape 不同形成不同的可定位、可排序、可检查 aggregation units。这不是穷尽式 2026 全量综述; 它是围绕当前 paper claim 的 closest-threat audit。

8 局限

当前结果仍不是 OSDI 或 NeurIPS 最终接收级完整证据。第一, field derivation 和 boundary recovery 的范围仍然有限。当前证据支持 deterministic mappings、label-derived rules 和部分 supervised boundary backends 能派生 stackable operation fields, 但不证明无监督或 LLM-

backed intent detector 已经解决, 也不证明一个 backend 能跨所有标签家族工作。AgentRewardBench looping 被简单 repeat_signal_change baseline 完全解释, 这说明每个 family 都需要 suitability、calibration 和 simple-baseline gate。

第二, R320 支持 profiler-localization claim, 但不支持人类效用 claim。R320 已经把 failure、safety、quality 和 human-boundary 问题转成 hidden-label ranking/localization benchmark, 并证明 operation-stack profiling 对 flat summary 和 fixed-session drilldown 有 accuracy/work/fragmentation tradeoff。Related-work/baseline audit 只把 grounding 转成可失败检查, 不增加新的实证结果。Protocol artifact 仍然有用, 因为它把现有 visible packets 和 hidden answer key 固化为 6 个 tasks、3 个 views、24 个 participant slots 和 144 个 trials 的 balanced analyst-study protocol, 并通过 visible-packet leakage check。但该 study 现在只是可选增强, 用来验证开发者或 agent analyst 是否更快或更准。本文当前不能推出开发者准确率、time-to-answer 或 productivity 改善, 也不能声称所有 intent boundary 都能被自动发现。论文主 claim 应固定为 profiler 本身在真实标注 trace 上的定位、排序、归因和 actionability, 而不是 human utility。

第三, 数据和互操作范围是轻量采样。本文避免同步完整测试集、图片归档和 gated 数据, 因此有些来源只覆盖前缀、子配置或 redacted rows。Chrome Trace Event 和 agent-session exchange 当前是 fixture-level smoke, 它证明 trace 可以作为 exchange container, 但不证明完整 OpenTelemetry/Chrome trace-ecosystem、Perfetto producer 或图像密集 benchmark 兼容。

第四, audit artifacts 是 claim-control surface, 不是额外实证结果。这些 audit artifacts 提高了论文表述、source path、负结果、related-work grounding 和 must-not-claim 边界的一致性, 但它们只能审计底层 empirical artifacts。如果底层采样或 oracle 本身有偏差, audit 能暴露并收窄 claim, 不能消除偏差。因此本文最终主张应固定为 two-abstraction mechanism 和 hidden-label profiler accuracy/work tradeoff, 而不是 universal fixed-session dominance、automatic anomaly detection 或完整 human utility。

9 结论

本文把 agent semantic profiling 收敛为两个核心抽象: operation 和 operation stack。Prompt、session、toolcall、process、syscall、plan、subagent、GUI action、安全标签和质量标签都只是 operation shapes 或 fields。Mapping 和 tagging 负责派生 fields, --view、--where 和 --stack 负责选择权重、查询子集和递归折叠形状, --trace-file 和 --operation-file 负责数据入口, --profile-spec 只把这些选择变成可复现实验配置, --deterministic-output 只把 profile artifact 时间戳固定为可复现值。在 15 个轻

表 6: Claim-centered result summary. 每行对应一个 paper claim 或一个 reviewer-expected counterpoint, 具体 provenance 保留在 evaluation ledger.

Paper question	Workload / oracle	Main evidence	Scoped conclusion
Can agent traces share one object model?	15 sampled public trajectory sources plus local-session exchange fixtures	Core public sources contain 42,590 operations, and ScaleCUA brings the smoke set to 47,590 operations. Agent-session and Chrome Trace Event round trips preserve 6 samples / 5 stacks on the fixture.	Heterogeneous trajectories and exchange traces enter the same operation layer. Trace formats are containers, not profiler abstractions.
Is stack depth a query rather than a prompt/session tree?	13,265 operations with depth sweeps, plus 16,741 AgentNet operations with profile-spec override and R321 predicate specs	The same operations fold from 9 dataset stacks to 57 phase, 226 tool/semantic, 455 action, or 3,757 fixed-session stacks. AgentNet stack override changes 608 stacks to 83 without changing input operations. R321 predicates select 729, 714, and 4,285 expected operations before folding.	Operation stack is a recursive query over fields. Fixed session is a useful drilldown field, not the default boundary.
Can fields be derived without adding a third object?	Held-out mapping splits, leave-dataset-out mapping, and OSWorld-Human supervised boundary probes	Held-out mapping improves compression from 14.049 to 19.091. Leave-dataset-out reduces stacks on 6/9 datasets with 0 negative regressions. OSWorld-Human learned boundary F1 reaches 0.7735 on held-out pairs.	Mapping/tagging/backends derive operation fields before folding. The claim is partial because calibration and simple-baseline checks remain family-specific.
Does recursive folding match human boundaries?	369 OSWorld-Human tasks, including 4,011 exact grouped-oracle operations	Grouped-depth stacks reduce 482 action stacks to 106 grouped stacks. Conservative human-group boundary F1 is 0.627 with precision 1.0 and recall 0.456.	The profiler can expose action-depth and human-group-depth views over the same sequence, but it does not fully recover latent intent boundaries.
Does the profiler solve real labeled problems beyond flamegraphs?	Six failure, safety, quality, and boundary tasks over 34,539 operations and 3,699 positives	Operation-stack packets are more selective than flat on 6/6 tasks, contain high-lift evidence on 5/6, and have higher selected recall than fixed-session on 5/6. They use lower selected work than fixed-session on only 2/6.	The supported value claim is an automated inspectability and localization tradeoff, with fixed-session preserved as a real low-work counterpoint.
Does profiling accurately localize and rank positives?	R320 scores 144 view/ranker policies with hidden labels over the same six tasks	Query-aware operation-stack top-5 groups inspect median 0.0937 work versus 1.0 for flat. They improve top-5 recall over fixed-session on 5/6 tasks and reduce median groups from 285.0 to 157.5.	Operation-stack profiling can be claimed as a faithful localization/ranking mechanism on real labeled traces, but not as a universal detector or single best hierarchy.
Are view, ranker, and mapping choices actionable?	R320 compares flat, fixed-session, dataset-native, raw-action, operation-stack, label-drilldown, width, query-aware, and oracle-upper policies; R322-R326 run Rust visible rank policies, feature ablations, and robustness probes	Query-aware ranking improves AP over width-only operation-stack ranking on 6/6 tasks. Rust operation-feature ranking improves semantic-stack AP on 5/6 tasks and first-positive work on 5/6 tasks; R325 finds 7 critical and 3 misleading feature instances. R326 global edge features beat width on 4/6 semantic and 5/6 coarse tasks, task-equal stays within 0.02 AP of weighted policy on 8/12 variants, and repaired policies improve AP on 2/3 misleading cases and first-positive work on 2/3 cases; 1/3 improves both.	The system should expose configurable stack fields, mapping rules, rank modes, operation-level rank features, global defaults, and task-specific rankers because different problems need different depths and drilldowns.
Are the profile specs reproducible with reportable local cost?	R327 and R328 repeat 76 tracked R300/R324/R326 profile specs over tracked operation JSONL inputs, for 152 Rust profiler invocations per run	R327 semantic profile hash, samples, and unique stacks are stable on 76/76 specs; default raw-byte determinism is 4/76 because JSON output includes generated_at. R328 enables deterministic output and reaches 76/76 semantic and raw-byte determinism, with median runtime 1.578s and p95 2.731s in that run.	The profiler has a reproducible offline profile-spec path and an opt-in byte-stable artifact mode with reportable local cost. This does not claim live eBPF overhead, human utility, or complete trace-ecosystem compatibility.

量采样的公开标注轨迹数据源上, 这个模型统一了 web、mobile、desktop、software、API、dialogue、failure、安全、human group 和 step-quality 轨迹, 并产生 flamegraph 之外的 stack tree、transition、quality、boundary、case packet 和 claim-audit reports. 这支持本文的核心机制 claim: agent profiling 的边界可以从 prompt/session/span tree 中释放出来, 成为 query-time operation-stack projection.

当前最强经验结论来自四类 oracle-rich 轨迹. OSWorld-Human 证明同一单动作序列可以折叠到人工 grouped-action 深度. AgentNet 证明大规模 human desktop step-quality 标签可以作为普通 operation fields 进入 profiling. AgentRewardBench 和 SATraj-OS 证明 failure、looping、side-effect、安全和 attack-type diagnostics 可以用同一机制表达. 6 个 label-hidden 任务进一步表明, operation stacks 相比 flat summary 更 selective, 并在多数任务上比 fixed-session 取得更高 selected recall. R320 把这个结论推进到 profiler accuracy benchmark: query-aware operation-stack top-5 groups 用 median 0.0937 work 取得 0.188 recall, flat summary 用 1.0 work 取得 full recall, fixed-session 用 0.0163 work 但 top-5 recall 只有 0.0233. 相对 fixed-session, operation-stack top-5 recall 在 5/6 tasks 上更高, 并把 median groups 从 285.0 降到 157.5. Related-work audit 则确认 pprof/tag pseudo frames、Perfetto SQL trace analysis、trace/span observability systems 和 public benchmark native views 都应保留为 baseline threats. Pareto frontier 又显示 operation-stack 在所有任务上 non-dominated, 但 flat 和 fixed-session 仍是真实 counterpoints. R320 的 per-task insights 最终把这些 proxy 结果压成六个任务的强项与反例: safety 和部分 step-quality 支持 selective triage, looping 更像 prevalence aggregation, side-effect 与 human-boundary 需要明确 ranker 和 stack depth. 因此

本文的 value claim 是 profiler fidelity、ranking/localization accuracy 和 actionable tradeoff, 而不是 detector、human utility 或 baseline dominance.

论文的新意不在单个 flamegraph, 而在两个抽象形成的统一接口. Mapping、tagging、profile specs、agent-session trace、Chrome/Perfetto-style trace exchange、boundary backends 和 reviewer evidence packets 都只围绕 operation fields 与 operation-stack queries 工作. 它们让用户和实验者能在同一批 operations 上选择不同 view、派生不同 fields、折叠不同深度, 并审计每个 claim 的证据来源. 下一步不应无差别增加数据集, 而应沿两个方向提高 claim 强度: 一是把 field-derivation backend 做到跨家族 calibration 并稳定胜过简单 sequence-derived baselines, 二是扩展 R320 的 hidden-label profiler accuracy benchmark 到更多 oracle-rich tool/API 和 mobile GUI families. Controlled human 或 agent analyst study 可以作为后续增强, 但不再是本文主 profiling claim 的前置条件.

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] OpenTelemetry GenAI semantic conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- [3] McGill-NLP WebLIX. <https://huggingface.co/datasets/McGill-NLP/WebLIX>.
- [4] OpenGVLab GUI-Odyssey. <https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey>.
- [5] WukLab OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- [6] xlangai AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.
- [7] McGill-NLP AgentRewardBench. <https://huggingface.co/datasets/McGill-NLP/agent-reward-bench>.
- [8] AI45Research SATraj-OS. <https://huggingface.co/datasets/AI45Research/SATraj-OS>.
- [9] AgentSuite tau-bench trajectories. <https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories>.
- [10] OpenGVLab ScaleCUA-Data. <https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data>.
- [11] OpenInference specification. <https://arize-ai.github.io/openinference/spec/>.
- [12] LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- [13] Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- [14] Arize Phoenix. <https://arize.com/docs/phoenix>.
- [15] AgentOps: Enabling Observability of LLM Agents. <https://arxiv.org/html/2411.05285v2>.
- [16] Brendan Gregg. The Flame Graph. <https://queue.acm.org/detail.cfm?id=2927301>.
- [17] Google pprof documentation. <https://github.com/google/pprof/blob/main/doc/README.md>.
- [18] Perfetto Trace Processor documentation. <https://perfetto.dev/docs/analysis/trace-processor>.
- [19] AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories.

ries. <https://arxiv.org/abs/2504.08942>.

[20] OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. <https://arxiv.org/abs/2404.07972>.

[21] OpenCUA: Open Foundations for Computer-Use Agents. <https://arxiv.org/abs/2508.09123>.

[22] Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. <https://arxiv.org/abs/2605.06230>.